

# Backend Dev 4

## Shop, Address, Review & Admin APIs

Detailed endpoint documentation for the marketplace's shop profiles, customer addresses, product reviews, vendor administration, and related product/catalog endpoints.

**Base URL**

`http://localhost:5000/api/v1`

**API prefix**

`/api/v1`

**Document version**

1.0 · 29 September 2026

# 1. Scope and conventions

This document describes the routes mounted by the backend app for shops, addresses, reviews, admin operations, products, and categories. The authentication service is owned by Backend Dev 1 / Queen and is not mounted in this app snapshot; protected routes expect a valid bearer token from that service.

## Contents

- |                                  |                                       |
|----------------------------------|---------------------------------------|
| 1. Conventions and authorization | 6. Admin                              |
| 2. Standard responses and errors | 7. Products                           |
| 3. Shops                         | 8. Categories                         |
| 4. Addresses                     | 9. Lifecycle and soft-delete behavior |
| 5. Reviews                       | 10. End-to-end examples               |

## Request conventions

- JSON endpoints use `Content-Type: application/json`.
- Product image create/update endpoints use `multipart/form-data`, with image field name `images`.
- MongoDB identifiers are 24-character ObjectId strings.
- Protected requests send `Authorization: Bearer <access-token>`.
- Query pagination is one-based. Product list defaults to page 1 and limit 12; admin lists default to page 1 and limit 20.

## Authorization summary

| Role     | Allowed operations                                                                                         |
|----------|------------------------------------------------------------------------------------------------------------|
| Public   | Read active shops, active products, categories, and visible product reviews.                               |
| Customer | Manage own addresses; create reviews; delete own reviews.                                                  |
| Vendor   | Register/update own shop; read own shop dashboard; create/update/soft-delete own products.                 |
| Admin    | List vendors/customers, change vendor active status, view analytics, create categories, delete any review. |

**Auth integration:** Exact token issuance, refresh, password, and account self-deactivation/deletion endpoints are defined by the auth owner. Protected route behavior requires `req.user.id` and `req.user.role`.

# 2. Response envelope and errors

All successful controller responses use this envelope:

```
{
  "success": true,
  "message": "Resource fetched successfully",
  "data": { }
}
```

Errors use the same stable top-level fields:

```
{
  "success": false,
  "message": "A useful error description",
  "data": null
}
```

| Status | Typical cause                                                 |
|--------|---------------------------------------------------------------|
| 200    | Successful read/update/delete.                                |
| 201    | Resource created.                                             |
| 400    | Validation, invalid ObjectId, duplicate key, malformed input. |
| 401    | Missing, invalid, or expired bearer token.                    |
| 403    | Authenticated role or ownership not authorized.               |
| 404    | Resource absent, inactive, or not publicly available.         |
| 500    | Unexpected server error.                                      |

## 3. Shop APIs

**POST** `/shops` Vendor only · protected

Register one shop for the authenticated vendor. One shop per owner is enforced.

| Body field                | Type       | Required | Notes                                       |
|---------------------------|------------|----------|---------------------------------------------|
| <code>name</code>         | string     | yes      | Trimmed, non-empty; maximum 100 characters. |
| <code>description</code>  | string     | no       | Defaults to an empty string.                |
| <code>logo</code>         | URL string | no       | Optional logo URL.                          |
| <code>contactEmail</code> | string     | no       | Contact address.                            |
| <code>contactPhone</code> | string     | no       | Contact number.                             |

```
{ "name": "North Star Goods", "description": "Home and lifestyle products", "logo":
"https://cdn.example.test/shop.png", "contactEmail": "hello@example.test", "contactPhone":
"+2348000000000" }
```

**Success:** 201; returns shop fields including `id`, `ownerId`, `isActive`, and `createdAt`. 400 if vendor already has a shop.

**GET** `/shops/:id` Public

Fetch an active shop by shop ID. The owning vendor must also be active and not soft-deleted. Response includes `productCount`. Returns 404 for unavailable shops.

**GET** `/shops/vendor/:vendorId` Public

Fetch a shop by the vendor's User ObjectID. Use this with `GET /products?vendor=:vendorId` to build a vendor storefront; there is no separate vendor-products endpoint.

**GET** `/shops/me/dashboard` Vendor only · protected

Returns the vendor's shop, `totalProducts`, and `activeProducts`. `totalOrders` and `totalSales` are currently `null` until the orders domain is integrated.

**PATCH** `/shops/me` Vendor only · protected

Partially update own shop. Allowed fields: `name`, `description`, `logo`, `contactEmail`, `contactPhone`. Submitted name/phone must not be empty; logo and email are validated when supplied. Returns 404 if no shop exists.

## 4. Address APIs

Address records reference the base user through `user`; multiple addresses per customer are supported. Customer shipping addresses currently require structured fields.

**POST** `/addresses` Customer only · protected

Create an address belonging to the authenticated customer.

| Field                  | Type    | Required | Notes                                                             |
|------------------------|---------|----------|-------------------------------------------------------------------|
| <code>fullName</code>  | string  | yes      | Recipient name.                                                   |
| <code>phone</code>     | string  | yes      | Recipient phone.                                                  |
| <code>street</code>    | string  | yes      | Street / delivery details.                                        |
| <code>city</code>      | string  | yes      | City.                                                             |
| <code>state</code>     | string  | yes      | State or region.                                                  |
| <code>country</code>   | string  | no       | Defaults to Nigeria.                                              |
| <code>isDefault</code> | boolean | no       | Defaults false; setting true clears other defaults for this user. |

```
{ "fullName": "Ada Customer", "phone": "+2348000000000", "street": "12 Market Road", "city": "Lagos",  
  "state": "Lagos", "country": "Nigeria", "isDefault": true }
```

**GET** `/addresses` Customer only · protected

List only the authenticated customer's addresses, ordered with default address first, then newest first.

**PATCH** `/addresses/:id` Customer only · protected

Update any subset of address fields. Supplied text fields must be non-empty where required; `isDefault` must be boolean. A true default clears the other defaults after this address saves successfully.

**DELETE** `/addresses/:id` Customer only · protected

Delete an address owned by the authenticated customer. Returns 404 when not found or not owned by caller.

**Address ownership:** The current Address schema requires `fullName`, `phone`, `street`, `city`, and `state`. User self-deletion/address-retention integration belongs to the auth lifecycle and is not an address route in this API.

## 5. Review APIs

**POST** `/reviews` Customer only · protected

Create one review per customer per product. Product and vendor must be active and available. Verified-purchase enforcement is deferred until the orders API is integrated.

```
{ "product": "64f000000000000000000001", "rating": 5, "comment": "Excellent product." }
```

`product` is a required Mongo ID; `rating` is an integer 1–5; `comment` is optional. Returns 201; duplicate user/product review returns 400.

**GET** `/reviews/product/:productId` Public

List reviews newest first for an active product with an active vendor. Reviews authored by inactive or soft-deleted customers are not publicly returned. User data is limited to ID and display name.

**DELETE** `/reviews/:id` Protected · owner or admin

Delete own review, or any review as admin. Product average and count are recalculated. Returns 403 when the caller does not own the review and is not admin.

**Rating behavior:** Ratings are recalculated from reviews belonging to active, non-deleted customers. Empty review sets produce average 0 and count 0.

## 6. Admin APIs

Every `/admin` route requires a valid bearer token with role `admin`.

**GET** `/admin/vendors?page=1&limit=20&status=active` Admin

List vendors with shop summary. Query: page (positive integer), limit (1–50), status (`active` or `inactive`; omitted for all). Results are newest first and include pagination metadata.

**GET** `/admin/customers?page=1&limit=20` Admin

List customers with basic profile and active status. Supports page and limit; newest first.

**PATCH** `/admin/vendors/:id/status` Admin

Activate or deactivate a vendor; body requires a boolean `isActive`. Vendor status and shop status are synchronized. Product-specific active state is preserved. Vendors with non-null `deletedAt` cannot be reactivated.

```
{ "isActive": false }
```

Returns 400 for attempts to reactivate a permanently soft-deleted vendor; 404 for an unknown/non-vendor ID.

**GET** `/admin/analytics` Admin

Returns counts for users, vendors, customers, shops, and active products. `totalRevenue` and `topVendors` are currently null pending orders integration.

## 7. Product APIs (storefront integration)

---

**GET** `/products` Public

List active products belonging to active, non-deleted vendors. Supports vendor storefront filtering; no duplicate vendor-products route is provided.

| Query                          | Type / constraints                              | Default             |
|--------------------------------|-------------------------------------------------|---------------------|
| <code>category</code>          | Mongo ID                                        | none                |
| <code>vendor</code>            | Mongo ID                                        | none                |
| <code>minPrice,maxPrice</code> | number $\geq 0$                                 | none                |
| <code>minRating</code>         | number 0–5                                      | none                |
| <code>search</code>            | string up to 100 chars                          | none                |
| <code>inStock</code>           | boolean string; true filters stock > 0          | false               |
| <code>sort</code>              | <code>newest,price_asc,price_desc,rating</code> | <code>newest</code> |
| <code>page</code>              | integer $\geq 1$                                | 1                   |
| <code>limit</code>             | integer 1–50                                    | 12                  |

Response data contains `products` summaries and `pagination` (`total`, `page`, `pages`, `limit`). Empty results retain the requested page and limit.

**GET** `/products/:id` Public

Fetch active product details plus up to four related products. Product's vendor must be active and not soft-deleted. Returns 404 when unavailable.

**POST** `/products` Vendor only · protected · multipart

Create product with required name, description, nonnegative price/stock, category ObjectId, and at least one image. Upload field is `images`, maximum five image files, 5 MB per file. Non-image MIME types are rejected. Image storage is handled by Cloudinary.

**PUT** `/products/:id` Owner vendor · protected

Update own active product fields and optionally replace images. Product updates do not permit changing vendor ownership. Replacement upload cleanup is failure-aware.

**DELETE** `/products/:id` Owner vendor · protected

Soft-delete own product by setting `isActive=false`; historical references remain intact. Reactivating its vendor does not reactivate this product.

## 8. Category APIs

---

**GET** `/categories` Public

List categories alphabetically with count of active products for each category.

**POST** `/categories` Admin · protected

Create a category. Required `name`; optional `icon` and `description`. Duplicate category names return 400.

## 9. Lifecycle rules and integration notes

- **Deactivation:** set `isActive=false`; keep `deletedAt=null`. Account and related records can be reactivated according to core auth policy.
- **Permanent soft deletion:** set `isActive=false` and a non-null `deletedAt`. Public shop/product/review queries exclude such users. Admin vendor activation rejects a non-null `deletedAt`.
- Shop and product records are not physically removed by vendor status changes. Product-specific `isActive` is preserved to avoid republishing previously deleted listings.
- Orders are outside this section and are not mutated by vendor status changes.
- Queen / Backend Dev 1 owns the shared User discriminator schema, account lifecycle endpoints, and auth middleware. This API assumes persisted `deletedAt` on that model.
- MongoDB integration regression test: set `TEST_MONGODB_URI` and run `npm test`. Without that variable, the integration case is explicitly skipped; pagination and compatibility tests run locally without MongoDB.

## 10. Example storefront flow

Fetch vendor shop profile, then fetch that vendor's products:

```
GET /api/v1/shops/vendor/64f000000000000000000001
GET /api/v1/products?vendor=64f000000000000000000001&page=2&limit=8
```

Example product-list data:

```
{
  "success": true,
  "message": "Products fetched successfully",
  "data": {
    "products": [
      {
        "id": "64f0000000000000000000010",
        "name": "Everyday Tote",
        "price": 8500,
        "image": "https://cdn.example.test/tote.jpg",
        "category": "Accessories",
        "categoryId": "64f0000000000000000000020",
        "vendorId": "64f0000000000000000000001",
        "inStock": true,
        "rating": 4.5,
        "reviewCount": 8
      }
    ],
    "pagination": { "total": 17, "page": 2, "pages": 3, "limit": 8 }
  }
}
```

```
}  
}
```